

3.0.pdf



Document Details

Submission ID

trn:oid:::47017:166303464

Submission Date

Sep 2, 2026, 3:16 AM UTC

Download Date

Sep 2, 2026, 3:19 AM UTC

File Name

GGULdmLIry1a.pdf

File Size

453.5 KB

16 Pages

6,180 Words

32,359 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

AI-Enhanced F1 Simulator

Individual Report

Author:

Supervisor:

Department of Computer Science

SEPTEMBER 2026

Word count: approximately 5,600 (body text, Sections 1–6)

Abstract

This report describes my part of the group project *AI-Enhanced F1 Simulator*. I worked on the Live Commentary feature with my teammate, Songyun Han. This is one of four AI features in the project.

I was mainly responsible for the part of the system that comes after the simulator detects a race event. I turned each selected event into a prompt for the model. I also controlled how the speech and captions were played, so that a busy moment in the race did not cause silence or overlapping speech. I built the tests used to check that this actually worked.

After a short introduction, my report has five main parts. I explain what I did in this project, and the ideas and new approaches behind the feature design. Then, the report describes how I built these features. This includes how I decided, when the system was busy, whether a new event should preempt the one already queued or be dropped. It also covers how I designed the prompts. After that, I set out how I tested the Commentary feature, and the methods I used to check that it worked correctly and reliably. Also, the report outlines how my teammate and I planned our work, including how we split tasks and scheduled the different phases of the work. In the end, I critically evaluate the work and look ahead to future work.

Author's Declaration

I declare that the work in this report was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in this report are those of the author.

SIGNED: DATE:

Table of Contents

1	Introduction and Context	1
2	Innovation and Creativity	2
3	Technical Development	3
3.1	Diagnosing the problem	4
3.2	First iteration: silent prefetch	4
3.3	Second iteration: a queued slot with priority replacement	4
3.4	Field-tagged prompt formatting	6
3.5	A related defect: context-budget headroom	6
4	Evaluation Skills	7
5	Project Management Skills	8
5.1	Fitting personal planning into the team's cadence	8
5.2	Personal task sequencing	8
5.3	Working within the group's interfaces	8
6	Critical Self-Evaluation and Future Work	9
6.1	Withdrawing bilingual commentary	9
6.2	The single-highest-priority-per-cycle limitation	9
6.3	Naming as an interface	10
6.4	Future work	10
A	Code Figures	12

1 Introduction and Context

Our group project is called *AI-Enhanced F1 Simulator*. It is a racing simulator with AI features, built on top of TORCS [1], an open-source driving simulator that supplies real-time data. Like any real-time racing simulator, TORCS produces telemetry much faster than a user can read it while driving. This is because vehicle position, damage, lap time, and the distance to nearby cars all update many times each second. In real motorsport, a broadcast commentator does a similar job. The commentator takes in far more information than a spectator has time to follow. They pick out the small part that really matters, and turn it into a sentence for spectators. This helps spectators quickly understand what is happening on track. In this project, the Live Commentary module plays this role. I worked on it together with my teammate, Songyun Han.

This feature serves two groups of users: drivers and spectators. A driver needs to keep almost all of their attention on the simulator screen. Commentary text on screen would pull this attention away, so we chose a channel that does not compete for visual attention: speech. With voice output, a spoken line about a rival closing from behind, or about the driver's own position changing, lets the driver understand how the race has changed. They never need to look away from the track [3]. For a spectator watching someone else drive, or watching the AI Driving Bot race on its own, the feature acts like the commentator a real broadcast would provide. This is something none of the project's other three features attempt to do. Using both text and speech together meets spectators' need to follow information that changes moment to moment. Neither drivers nor spectators want a system that talks constantly or repeats itself. This is why, as later sections describe, a large part of my own work was not about generating text. It was about selecting and pacing it.

Live Commentary is one of three features that need the same real-time simulator data. The other two are the Race Engineer and the Telemetry Coach. Instead of each feature opening its own connection to TORCS, the project built one shared layer to collect this data. This telemetry arrives through a read-only UDP interface. A single `TelemetryService` parses it and stores it in a thread-safe `TelemetryStore`. Any of the three features can read from this store without competing for the same socket. The Live Commentary is built on this shared store. However, its downstream components, including event detection, prompt construction, and output scheduling, are all private to Commentary. Subsequently, it passes through a shared component called the `ModelBroker`, which handles model calls. My work primarily focuses on the private downstream components of the commentary feature. I will describe what happens when a chosen race event transforms into a bounded prompt, how the commentary output is properly scheduled to prevent unwanted silences or interruptions, and also how I tested that pipeline.

Figure 1 shows this structure from top to bottom: at the top level, a shared store collects data; then the pipeline branches into three separate features. My contribution lies in the specific path of Commentary, which diverges from the other two before routing back through the shared `ModelBroker`, and in the private output-scheduling pipeline that follows it. This pipeline is described in full detail in Section 3.

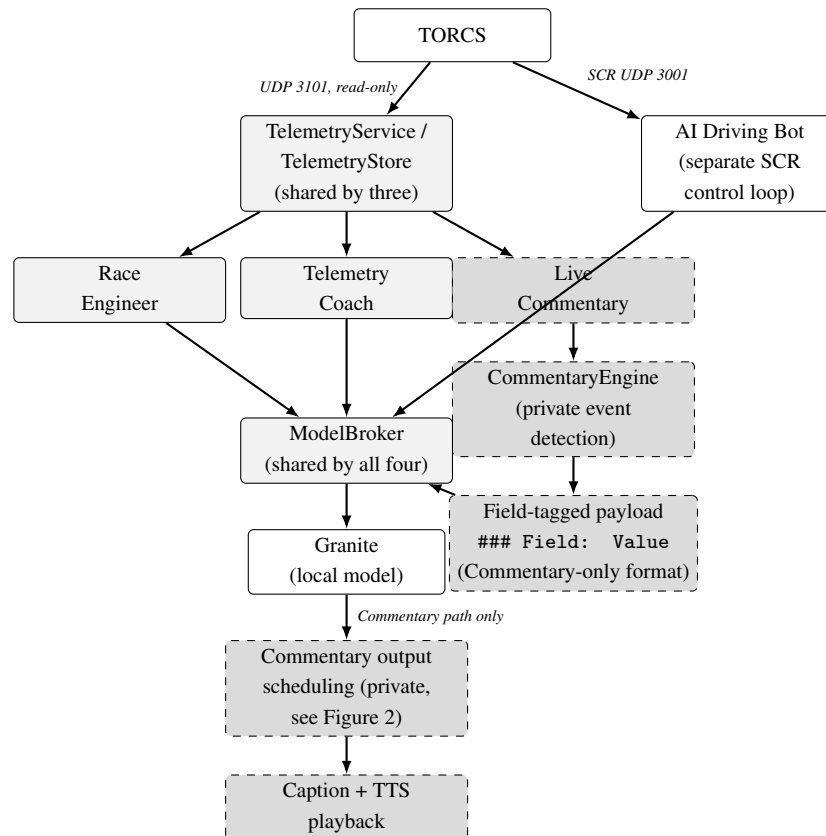


Figure 1: The diagram contrasts the shared workflow (solid boxes) with Commentary’s own private workflow (dashed boxes): telemetry acquisition is shared by three features, while the model broker is shared by all four. SCR (Simulated Car Racing) is the TORCS control interface used by the AI Driving Bot.

Unlike the AI Driving Bot, whose output is part of the control loop and actually drives the car in the simulator, the output of the Commentary feature does not enter the control loop. Even if an anomaly occurs, such as an unusually slow model invocation, it merely results in a single missed event in the race, without causing the simulation to fail. This limited role is exactly what makes the feature acceptable. This makes it both safe and practical to put a language model, whose response time is on the order of seconds and whose outputs lack guarantees, between the user and simulator. This bounded role is not limited to racing. Whenever information is generated too quickly for a human to keep up, a language model can summarise it into something a human can easily understand, without interfering with core operations. Live sports data, an industrial monitor or a real-time dashboard are all possible examples. Sections 3 and 4 discuss this boundary further. They explain how the system prevents a pending or delayed model response from blocking the rest of the telemetry pipeline. I built some tests to check whether the mechanism continues to work properly under actual faults and simulated faults.

2 Innovation and Creativity

In the Commentary section, Han and I handled separate stages of the pipeline: Han focused on the early architectural design, while I was responsible for later feature development and optimisation on the basis of his work. What Han built handles event detection, which analyses incoming telemetry changes to identify whether they are worth commentating on—that is, whether they qualify as an event. This consists of three parts: a sliding window event detector, an importance-based hierarchy among various events, and cooldown and de-

duplication policies. In short, it answers the question: should this particular event be narrated? Section 4 later proved the effectiveness of this design.

However, this design does not solve another underlying problem: assuming an event is considered worthy enough to tell, should it be played at that moment? Consider the following scenario: Commentary requests that Granite provide a sentence and is about to commentate on it. However, a higher-priority event occurs before the first sentence finishes playing. How should the system handle this? In an earlier version of the pipeline system, the identical segment of code dealt with both issues. In other words, the system viewed “worth narrating” and “worth playing right now” as a single decision. Of course, this is not an issue during the quiet periods when few events are triggered. Yet, once the on-track action heats up with heavy multi-car battle, continuous off-tracks or recurring collisions, this mechanism starts breaking down. Unfortunately, this happens at the very moment when commentary matters most for the user.

Early in the project, I noticed this problem during a demo and immediately brought it up to my teammate. After reproducing that scenario, we both realised that they are actually two independent problems: “whether an event is worth commentating on at all” and “whether it is worth commentating at that exact moment.” We discussed an architectural solution: introducing a dedicated output scheduling layer for the commentary system, which is driven by two small state machines. I developed and implemented this layer on my own, in the form of an interrupt/queue playback mode, located completely behind Han’s event/interval/hybrid trigger logic. This allowed me to think about two issues separately: is this worth speaking out about; and how will the queued speech behave upon the arrival of something more significant. Section 3 will further explain this scheduler. In my view, the core insight was recognising that these were two independent issues, which a single state machine could not resolve cleanly. This problem was far from obvious at first and only became exposed during the high-stress live racing scenarios. This experience highlights the value of continuous live testing, as well as the benefit of open, collaborative discussions with my teammate.

I have an example of the same concept, which is not accepting anything as true just because it already exists. It relates to the method for the given event to reach the model. In the beginning when we created this component, we just wrote the payload of the selected event in the form of JSON, and transmitted it to the model without making any modifications. We did not give much thought to the format itself. Later, noticing occasional model latency issues, we started analysing the possible factors behind it. We came to realise that the data payload sent to the model could be part of the problem. It impacted the token cost of any request, as well as the quality of the output produced by the model. Brackets, quotations, and indentation in JSON are not indicative of the race. However, the model still needs to read them and may repeat them later in the output. After identifying it as a real issue and not merely a guess, I looked up how others tackled this task: converting structured data into prompts. Turning to the literature, we found a similar commentary system. What struck us was just how remarkably clever their approach was and it did give us the inspiration we needed. Section 3 explains this further. My approach here is similar: using a plaintext format based on field labels rather than JSON. For instance, `### Speed change: +38 km/h` instead of `"speed_delta": 38`. The format contains only the fields relevant to each event type, instead of always sending the whole payload.

3 Technical Development

While the group report offers an overall view of the functionality of the Commentary feature, this section will provide more details about my contribution, which focuses on the output scheduler and prompt-building. To begin with, I will introduce the motivation of building this scheduler and the two design iterations of its

architecture. Next, I will explain the process of building the prompt structure constructed for each request. Finally, I will introduce an issue that I discovered during testing, which involved a context-budget defect, and how I fixed it.

3.1 Diagnosing the problem

This subsection describes the scheduler's position in the overall commentary pipeline and its functional purpose. The upstream part involves choosing an event to determine whether a telemetry update is worth narrating; this is the function that Han made, mentioned in Section 2. The downstream involves keeping the two outputs synchronised, which are the caption displayed on the shared dashboard and the audio message played via text-to-speech (TTS) service. Its primary function is to ensure synchronisation between the generated text captions and the voice playback when an event is recognised.

The scheduler could only maintain a single request at any given point in time in its very first iteration. Incoming events either waited for the completion of the ongoing request or were cancelled under interruption mode. The second form of this behaviour led to the issue. Whenever the scheduler cancelled an outdated request to initiate a new one, a dead-air issue arose. Neither subtitles nor audio playback was produced, until Granite generated the first token of the new sentence, which could take some time. Ironically, this problem occurred during periods of dense events activity—the exact moments when the user most needed the commentary. This severely degraded the user experience.

3.2 First iteration: silent prefetch

My first iteration primarily addressed the silence issue that was previously mentioned. Instead of cancelling the ongoing request, my new scheduler design allows the second event to trigger a silent generation request, which is then dispatched to the `ModelBroker`. In this way, the ongoing commentary line plays out smoothly without being abruptly interrupted. “Silent” here meant the request ran in the background, and its tokens built up there. Nothing was sent to the `WebSocket` or the TTS service until it was that request's turn. This meant the dashboard and the TTS service were never left empty while a new line was still generating, but it created a new problem. If a third event occurred at this point, it would be dropped because the scheduler still had only one pending slot. This raised the problem to a new level: handling multi-event congestion rather than two-event conflicts. Figure 3 shows the function this silent request – the prefetch – became: `_start_prefetch()` in `middleware/runtime.py`. It cancels any earlier prefetch that is still running, before starting a new one for whatever event is now queued.

3.3 Second iteration: a queued slot with priority replacement

The final version of the scheduler no longer interrupted the actively playing sentences. Module-level state in `middleware/runtime.py` tracks two things separately: `_commentary_task`, the coroutine that is currently generating or already playing, and `_pending_decision`, at most one event waiting to take over once the current line finishes. A new event that arrives while the system is busy goes through `_queue_event()` (Figure 4). If its priority is lower than whatever is already in `_pending_decision`, it is dropped outright. Otherwise, it replaces the pending slot, and `_start_prefetch()` is called again, so the silent generation stays in sync with whatever is now pending. This design – where a lower-priority event is discarded, rather than entering a queue – was not something I decided alone. Following our supervisor's suggestion to use a queue (Section 4), Han

and I determined this design together, through discussion, after testing an earlier version. We noticed that, during busy periods, a queue that accepted every collected event fell further and further behind the live race. We agreed that dropping a lower-priority event was better than the scheduler eventually describing something that had already happened several seconds earlier.

After Han and I had settled on dropping rather than queuing, I looked for how this problem had been studied elsewhere. I found a study about a data stream management system, and noticed that the system built by Tatbul's team faced a similar question [7]. When an overloaded stream processor cannot keep pace with incoming tuples, it should make decisions on which data to drop. Rather than dropping tuples at random, their system strategically drops the tuples which minimise the quality of service loss from the output stream. From their study, I first learned the technical term for this problem: load shedding. It means deciding what to discard when a system cannot keep up with everything arriving at it. My own logic of `_queue_event()` is a small-scale drop decision: retaining high-priority events, dropping low-priority events. The event priorities are what make this a principled drop rather than an arbitrary one. That is what I learned from the literature, and I applied it to my own design.

When the frontend reports that playback has finished, `_advance_queue()` (Figure 5) releases the pending slot. If the system is not in queue mode, or nothing is pending, it simply clears the busy flag and goes idle. Otherwise, it takes over the pending decision and its prefetch task. It waits for that prefetch to finish, if it has not already, before broadcasting the result as the next caption and audio clip, through `_broadcast_silent_result()`.

Figure 2 sets out the finished pipeline described above, from a selected event to the caption and audio that eventually reach the user.

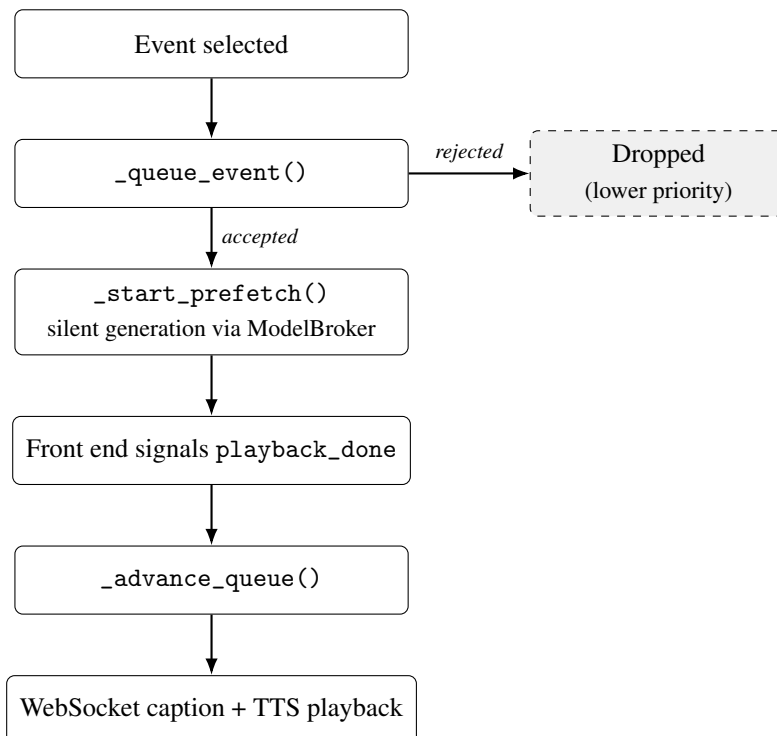


Figure 2: The finished output-scheduling pipeline, from a selected event to the caption and audio delivered to the user. The dashed box marks the one point where an event is discarded rather than carried forward.

3.4 Field-tagged prompt formatting

The field-tagged rendering described in Section 2, which writes each piece of event data as `### Field: Value` rather than as JSON, was the outcome of joint experimental testing and reflection by Han and myself together. We looked into how large language models process input and how they consume tokens, and found that the original JSON format was probably increasing tokens consumption and degrading generation quality. I then looked for how other people had solved a similar problem, and found Cook and Karakuş's paper on fine-tuning a language model to generate football commentary from structured match-event data [6]. Their system formatted incoming model inputs by utilising `###` (hash) prefixes so that the model could clearly distinguish the boundaries of each event data point. This approach significantly improved the model's ability to interpret incoming event feeds, so I adopted it in our implementation.

It should be noted here that the approach used in our project is different from that mentioned in the literature. They annotate the training samples with the `###` prefix markup. My prefixing convention was applied to data payloads dynamically assembled per request, feeding into a model without any fine-tuning for my commentary feature. Since this particular project was unable to fine-tune the Granite model, the same formatting idea does a different job here. Rather than adapting the model during training, I introduced the structural pattern at inference time. Of course, this approach relies on the model's baseline capacity to interpret structured context—an area which the Granite model already reliably handles.

3.5 A related defect: context-budget headroom

The context-budget flaw was not a problem I had foreseen during initial design. It appeared during one of my manual tests (Section 4), where a request that could have passed was declined instead. Each request prompt is generated by combining the persona, the event payload, and a small portion of recent commentary history using `ContextManager`. Its implementation is simple: it estimates how many tokens this text will consume—about one token for every four ASCII characters and one token for every two CJK (Chinese, Japanese, and Korean) characters. It doesn't employ any actual tokenizers since it would take too much time to run it on every round. The heuristic is merely an approximation. During one test, it occasionally underestimated the length of a long history, miscalculating it as barely fitting the model's context limit. When this happened, the request was directly rejected.

The initial version of the budget left no buffer to absorb this kind of errors. The token budget of history had been calculated as `max_context_tokens` minus `max_response_tokens`, with no residual even when the estimate was incorrect. It was a mistake that was easy to make during the first attempt because when it went wrong, it appeared to be a modelling problem rather than an error in the budgeting process. My solution was simple: the fixed safety margin (`safety_margin_tokens`) is subtracted from the history budget prior to the trimming. Thus, the heuristic estimation does not have to be perfect and the request should succeed. In a subsequent test, I reproduced this scenario on purpose to verify whether my fix was effective. In summary, what I did for this feature—the implementation of the pending-slot scheduler and the context budgeting safeguards—provided effective solutions to the two kinds of question I often ran into. The former was an architectural issue, while the latter was an error for heuristic estimation. Together with the prompt formatting challenge, these experiences truly underscored the importance of referencing academic research to me. These issues were not explicitly detailed in our group report, so it was necessary to clarify them all properly in this report.

4 Evaluation Skills

My teammate, Han, created a test plan on the subject of Commentary that outlined what categories of events should be included as well as the pass or fail standards of detection accuracy and prioritisation behaviour. My task was to follow this plan to test all implemented features and identify existing issues. To achieve this, I developed a suite of test scripts, including an automated validation that feeds pre-constructed telemetry frames into the event detector and confirms it returns the correct event type. From my perspective, this obviously boosted testing efficiency by removing the need to manually rerun the full race after each code adjustment. I also created a monitoring script which recorded all events reported by the system including timestamps, followed by another script that compared these logs against my manually labelled event list to evaluate the detection accuracy. In addition, I verified that what the system claimed it had detected really corresponded to events that actually occurred while driving the car in the simulator, not relying on the log only. This is important because, while a language model can produce text that sounds entirely reasonable, the generated output may completely lack supporting data. This is a phenomenon called hallucination [2].

Evaluation of this feature did not only come from my own test plan. One design decision came directly from formative feedback from our supervisor. Near the end of the project, as we were preparing for our planned user evaluation, our supervisor asked us to demo the feature during a group meeting. She found the immediate-interruption delivery jarring. She did not like how a new event cut off the line already playing, and she suggested handling short bursts of events with a queue instead. This prompted me to redesign the mechanism, away from immediate interruption and towards queuing new lines instead of cutting off the current one. Once I started building it, though, I found that a queue accepting every event fell behind the live race during busy periods. So I built a single-slot mechanism with priority-based replacement instead of queuing every event; the choice to drop lower-priority events rather than queue them was one Han and I reached together, as Section 3 describes. This was formative feedback driving a design iteration, not a result from our planned user evaluation.

I ran through the whole plan by hand four times during development, not just once at the end. In each of the first three rounds, I found one real problem. One problem was a repeated commentary output, caused by placing the deduplication check after the WebSocket broadcast. Another problem was failing to properly set the trigger threshold for the `pace_surge` event, causing it to be triggered even on marginal speed increases. The third was the context-budget rejection I mentioned earlier in Section 3. Every single time, I traced the problem back to the exact piece of code causing it, fixed it, and re-ran that part of the plan before moving on. It was not until the fourth round of testing that the whole plan passed without errors. At that point, I deemed it compliant with the acceptance criteria defined by Han.

Throughout this process, the most effort-consuming task was checking, by hand, whether passing test cases were actually correct. In certain cases, events logged by the script appeared correct on the surface, but were not actually valid [4]. For instance, the script logged a contact event but completely missed an off-track excursion event, which happened at the same time. Such an omission was invisible in the log output itself. In order to verify this, I took accurate starting and ending timestamps of all tests and made a video of the entire session. Then I examined the CSV file of the monitoring script line by line and correlated each recorded event with the exact point in the recording using the timestamps. This was how I assured myself that the events that the script reported were indeed the ones that occurred on-screen, rather than simply assuming them. Using this method, I identified an additional issue: when two events with the same priority occurred in the same detection window, the detector retained only one and silently dropped the other.

By the time all testing was completed, this feature had passed every criterion in Han's test plan and had

been verified against real footage rather than a self-consistent log. Furthermore, it held up well under both real faults and faults I created on purpose. These results showed me that the commentary feature was mature and reliable.

5 Project Management Skills

The group report already describes project management at the team level. This section covers how I personally planned, managed, and took part in developing the Commentary feature with Han, and how this two-person, collaborative way of working shaped my own methods.

5.1 Fitting personal planning into the team's cadence

Our development followed the SCRUM methodology [5]. Each week was run as a sprint. Every Monday morning, our team attended a weekly meeting organised by our academic supervisor, Professor Ruzanna Chitchyan, where we reported our progress and gathered her guidance. Straight after that, the whole team held an internal planning meeting that same afternoon. The team gathered to distribute sprint tasks, discuss shared issues of development and track each feature's progress. During this meeting, Han and I further divided the commentary tasks and logged the work to be done onto the team's shared Jira board, which I created to ensure visibility of development tasks and sprint progress. Every Friday, we sent a progress update covering the entire project to both our academic and IBM supervisors, allowing them to assess our progress in advance of the upcoming meeting. I had to plan my own weekly work carefully, so that I always had something worth reporting at the group meeting. This let me get feedback from our supervisor in good time, and use it to improve my own design.

5.2 Personal task sequencing

My work was organised in a layered manner: I prioritised an end-to-end working pipeline across each commentary layer before enriching it with specific details. I initially implemented a little middleware service as well as a `ContextManager`, which would transform incoming real-time telemetry data into commentary text, paired with a basic frontend for subtitles and voice output. This allowed us to have a functioning commentary system as early as Week 2, although it was still quite unrefined. Having this tangible visual and auditory feedback, I was able to more effectively improve the core elements that directly impacted commentary quality, including prompt design, context budget and interrupt/queue scheduling, which I described in Section 3. Because we had multiple features and several shared code files, I kept my own work on separate branches pulled from main (`voice2.0`, `voice6.0`, `commentary4.0`, and so on), rather than committing straight to main. This prevented my modifications from interfering with other teammates' code. Following the team's workflow, I only merged a branch back into main after Han and I had reviewed it. This also matched the team's rule: no development branch is merged into main until it has been thoroughly verified on more than one device.

5.3 Working within the group's interfaces

My individual work was largely built upon the foundational infrastructure developed by other teammates: Han's event-detection logic, the shared `TelemetryService` and the shared `ModelBroker` used by every feature. Con-

sequently, I dynamically adjust my development approach to keep pace with continuous updates to upstream modules.

One example came up while I was refactoring `ContextManager`. I wanted to rename the `commentator_persona` field to something clearer. I decided not to. That exact name was already part of the API used by the dashboard configuration panel that Shangqing Li had built for the Race Engineer. Renaming it would have broken part of a system I did not own, and could not easily test myself. So I kept the name, and improved the code around it instead. This is a small example, but I think it is a good one: treat any interface another teammate's code depends on as fixed. Even when I can see a better approach myself, I should still discuss it with the teammate first, before making a change.

The same thing happened at the output end. My caption and voice frontend, along with a separate Electron overlay, was eventually replaced. At that time, our team had reached a consensus that all features would share a single HTML dashboard as the frontend, so I moved my playback logic into it, and deleted the old, separate Commentary page. Working in a team project required my individual design and development work to adapt dynamically to the team's latest decisions.

6 Critical Self-Evaluation and Future Work

Reflecting on my overall development experience in this group project, there are still a few things I would like to improve. One is withdrawing bilingual commentary, a decision Han and I made together after finding a problem and testing it ourselves. Then, it is the single-highest-priority-per-cycle limitation, which only surfaced during testing (Section 4) and traces back to an assumption I accepted without questioning it. One more is a smaller, ongoing naming problem across the pipeline, which better coordination early on could have avoided.

6.1 Withdrawing bilingual commentary

Commentary feature, in its early version, supported spoken narration in two languages, English and Chinese. But neither the Chinese text subtitles nor the audio commentary met the quality standards that Han and I set. The Chinese commentary often had errors that changed the meaning, or used the wrong words. At the same time, the Chinese TTS voice sounded unnatural, and was far behind the quality of the English voice. After discussing this with Han, we agreed that neither of us was happy with the design. We also showed it to the team, and the feedback was consistently poor too. So we dropped it, and kept only the English version. If bilingual, or multilingual, commentary is developed further in future, I would first set up a formal translation-quality test. I would also actively try new voice packs, in Chinese or other languages, once better ones become available.

6.2 The single-highest-priority-per-cycle limitation

By comparing the test recordings against the detection log, Section 4 identified a notable issue where an off-track excursion happened in the same cycle as a contact event. In the detector, `contact`, `off_track` and `position_change` all share the same priority value. So when two of them fire in the same cycle, only one ends up in the log, and checking that log against the recording cannot show which event was dropped. Which one survives is decided by order alone: because `contact` is appended before `off_track`, `contact` always wins and `off_track` is silently dropped. The result is that `off_track` recall drops exactly in the moments when

both events happen together. I originally treated it as a simple priority rule without verifying whether these priorities were actually equal, and I did not realise the issues it caused until I saw the test results. Whenever two events with equal priority occur in the same cycle, this design produces an incomplete event log. If I were to redesign this, I would separate event recording from event narration, logging every detected event into a structured log. The group report's future-work section suggests the same direction for Commentary, and it is what I would work on first if this project were to continue.

6.3 Naming as an interface

A smaller problem that kept coming up was naming. Han's trigger modes are called `event`, `interval` and `hybrid`. The output-scheduling modes I built are called `interrupt` and `queue`. Both sets of names make sense on their own. But anyone reading the code without this report has no way to know that they describe two different layers of the same pipeline, not two competing versions of the same thing. More than once during integration, a teammate asked me which of the two settings controlled which part of the behaviour. In hindsight, agreeing on a shared naming convention with Han, before either set of names was decided, would have cost almost nothing at the time. This would have made the two-layer structure clear from the code itself, instead of only from a report written afterwards. For example, we could have used a shared prefix for trigger settings, and a different one for scheduling settings.

6.4 Future work

Beyond what I have already mentioned, I see two clear next steps for Commentary. The first is a visualisation window built directly into the simulator, to display the caption there, instead of only on the separate shared dashboard. The second builds on this: an automated test script. Using screen image recognition on this embedded window, the script could automatically check the events it recorded against what actually happened, instead of relying on the manual timestamp-and-recording check from Section 4. If new feature modules are added to the project in future, this kind of automated check would make testing far more efficient.

References

- [1] B. Wymann, E. Espié, C. Guionneau, C. Dimitrakakis, R. Coulom, and A. Sumner, “TORCS, The Open Racing Car Simulator, v1.3.5,” 2013.
- [2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.
- [3] M. R. Endsley, “Toward a theory of situation awareness in dynamic systems,” *Human Factors*, vol. 37, no. 1, pp. 32–64, 1995, doi: 10.1518/001872095779049543.
- [4] J. S. Santillana, “Precision is not faithfulness: Coverage-aware evaluation of grounded generation with a complete oracle,” arXiv preprint arXiv:2606.09376, 2026.
- [5] A. Srivastava, S. Bhardwaj, and S. Saraswat, “SCRUM model for agile methodology,” in *2017 International Conference on Computing, Communication and Automation (ICCCA)*. IEEE, 2017, pp. 864–869.
- [6] A. Cook and O. Karakuş, “LLM-Commentator: Novel fine-tuning strategies of large language models for automatic commentary generation using football event data,” *Knowledge-Based Systems*, vol. 300, art. 112219, 2024, doi: 10.1016/j.knosys.2024.112219.
- [7] N. Tatbul, U. Çetintemel, S. Zdonik, M. Cherniack, and M. Stonebraker, “Load shedding in a data stream manager,” in *Proceedings 2003 VLDB Conference*, Morgan Kaufmann, 2003, pp. 309–320.

A Code Figures

```

async def _start_prefetch(decision: CommentaryDecision) -> None:
    ...
    global _prefetch_task
    if _prefetch_task and not _prefetch_task.done():
        _prefetch_task.cancel()
    t, r = telemetry_store.latest()
    request_id = str(uuid.uuid4())

    async def _run():
        return await generate_commentary(
            t, r, event_payload=decision.payload, history_mode="summary",
            request_id=request_id, silent=True,
        )

    _prefetch_task = asyncio.create_task(_run())

```

Figure 3: `_start_prefetch()`, `middleware/runtime.py`: cancels any in-flight prefetch and starts a new silent generation (`silent=True`) for the currently queued event.

```

async def _queue_event(decision: CommentaryDecision) -> None:
    ...
    global _pending_decision
    new_priority = decision.event.get("priority", 0)
    if _pending_decision is not None and new_priority < _pending_decision.event.get("priority", 0):
        return
    _pending_decision = decision
    await _start_prefetch(decision)

```

Figure 4: `_queue_event()`, `middleware/runtime.py`: a lower-priority event is dropped; anything else replaces the single pending slot and re-triggers prefetch.

```

async def _advance_queue() -> None:
    ...
    global _pending_decision, _prefetch_task, _commentary_task, _commentary_priority, _playback_busy

    if commentary_engine.config.interrupt_mode != "queue" or _pending_decision is None:
        _playback_busy = False
        return

    decision = _pending_decision
    _pending_decision = None
    prefetch = _prefetch_task
    _prefetch_task = None
    _commentary_priority = decision.event.get("priority", 0)

    async def _release():
        try:
            result = await prefetch if prefetch is not None else None
        except asyncio.CancelledError:
            result = None
        except Exception as e:
            log.warning(f"Queued commentary prefetch failed: {e}")
            result = None
        if result is not None:
            await _broadcast_silent_result(result, event=decision.event, payload=decision.payload)

    _commentary_task = asyncio.create_task(_release())

```

Figure 5: `_advance_queue()`, `middleware/runtime.py`: releases the pending slot once the frontend signals `playback_done`, waiting on its prefetch task if it has not finished yet.